

Where:

- y =Predicted value
- x =Input feature
- m =slope(weight)
- b =Intercept(bias)

Example:

- $\text{Salary} = 5000 * \text{Experience} + 20000$
- If experience=5 yrs
- $\text{Salary} = (5000 * 5) + 20000 = 45000$

Why do we need a cost function?

- Initially, the model guesses values for slope(m) and intercept(b).
- These guesses are usually wrong.
- So we need a way to measure:
- "How bad is our prediction?"
- This measurement is called the Cost Function.

Mean Squared Error(MSE)

- The most common cost function for linear regression is MSE.

Formula

- $\text{MSE} = \frac{\text{Error}^2 + \text{Error}^2 + \dots + \text{Error}^2}{n}$
- $\text{Error} = \text{Actual value} - \text{Predicted value}$ #diff btw this called the error

Introduction to Regression

- Regression is a supervised ml technique used to predict a continuous numerical value based on 1 or more input features.
- The goal of regression is to find the relationship btw independent variables(features) and a dependent variable(target).

Example: 🏠

- Predicting house prices
- Predict employee salaries
- Predict stock prices
- Predict sales revenues

Linear regression

- It is the simplest regression algorithm that models the relationship btw ip and op variables using a straight line.
- It assumes that a linear relationship exists btw the independent variable and dependent variable.

Objective

- Find the best fitting straight line that minimizes prediction errors.

Equation of Linear Regression

For one feature:

- $y=mx+c$

Steps

1. Find the error for each prediction
2. Square each error.
3. Add all squared errors.
4. Divided by the total no of observations.

Example

Actual	Predicted	Error	Error ²
10	8	2	4
15	14	1	1
20	18	2	4

- $MSE = (4+1+4)/3=3$

Gradient Descent

- It is a method used to find the best line by reducing the prediction error step by step.

Real time example:

- Imagine you are on top of a mountain and want to reach the lowest point.
 1. Take a small step downward.
 2. Check if you're getting lower.
 3. Keep taking small steps down.
 4. Eventually, you reach the bottom.

```

#Gradient
dm=(-2/n)*np.sum(X*(y-y_pred))    #how much to change slope
db=(-2/n)*np.sum(y-y_pred)         #how much to change intercept

#Update parameters
m=m-learning_rate*dm
b=b-learning_rate*db               #move toward lower error

print("Slope(m):",m)
print("Intercept(b):",b)

```

```

Slope(m): 2.0048610156782556
Intercept(b): 0.9857080211211781

```

▼ OLS(Ordinary Least Squares)

- When we draw a regression line, there can be many possible lines.
- OLS helps us find the best line.

How?

1. Predict values using a line.
2. Compare predicted values with actual values.
3. Find the error.
4. Square all errors.
5. Add them together.
6. Choose the line with the smallest total error.

- This is exactly how gradient descent works.

In ML:

- Start with random values for slope and intercept.
- Make predictions.
- Calculate the error using cost function.
- Adjust the values slightly to reduce the error.
- Repeat until the error becomes very small.

```
3]: import numpy as np

#Dataset
X=np.array([1,2,3,4])
y=np.array([3,5,7,9])

#Initial values
m=0 #start with random guesses.
b=0

learning_rate=0.01 #Controls how much the model updates each step.
epochs=1000 #no of iterations

n=len(X)

for i in range(epochs):

    #Predictions
    y_pred=m*X+b #Current line equation
```

```
import pandas as pd
from sklearn.linear_model import LinearRegression

df=pd.DataFrame({
    'Experience':[1,2,3,4,5],
    'Salary':[30000,35000,40000,45000,50000]
})

#input feature(X)
X=df[['Experience']]

#Target variable(y)
y=df['Salary']

#Create model
model=LinearRegression()

#Train model(OLS happens here)
model.fit(X,y)

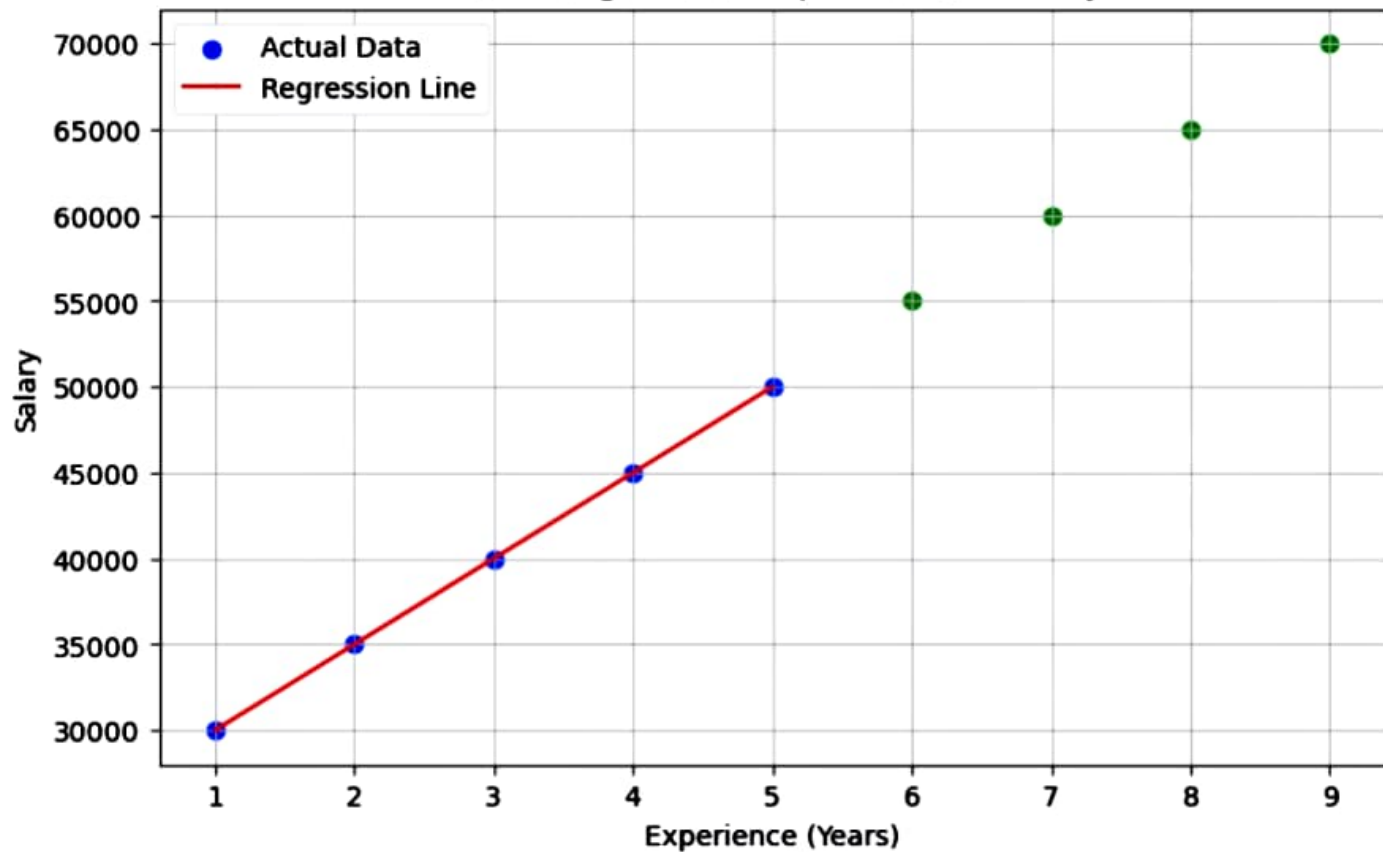
#Predict salary
predictions=model.predict(X)

#Results
print("Slope:",model.coef_[0])
print("Intercept:",model.intercept_)
print("Predicted Salaries:",predictions)

#When fit() runs,OLS automatically calculates teh best slope and
```

```
plt.grid()  
plt.show()
```

Linear Regression: Experience vs Salary



Slope: 5000.000000000001
Intercept: 24999.999999999996
Predicted Salaries: [30000. 35000. 40000. 45000. 50000.]

```
new_experience=pd.DataFrame({'Experience':[6,7,8,9]})
```

#Predict salary for new values

```
new_predictions=model.predict(new_experience)
```

```
print("Predicted Salaries for 6,7,8,9:")  
print(new_predictions)
```

Predicted Salaries for 6,7,8,9:
[55000. 60000. 65000. 70000.]

```
import matplotlib.pyplot as plt
```

```
plt.figure(figsize=(8,5))
```

Original data points

```
plt.scatter(df['Experience'], df['Salary'], color='blue', label='Actual Data')
```

Regression line (trained data)

```
plt.plot(df['Experience'], predictions, color='red', label='Regression Line')
```

New predictions

```
plt.scatter(new_experience['Experience'], new_predictions, color='green')
```

Labels

```
plt.xlabel("Experience (Years)")
```

```
plt.ylabel("Salary")
```

```
plt.title("Linear Regression: Experience vs Salary")
```

```
plt.legend()
```


Multiple Regression

- It is a regression technique that uses 2 or more independent variables to predict a dependent variable.

Example

- Predicting a house price using:
 - Area
 - No of bedrooms
 - House age
- Instead of one feature, we use multiple features.

Example Dataset

Area (sq ft)	Bedrooms	Age (years)	Price
1000	2	10	300000
1200	3	8	350000
1500	5	5	450000

Here:

- Features=Area,bedrooms,Age
- Target=Price

```
#Multiple regression code
```

```
import pandas as pd
```

```
from sklearn.linear_model import LinearRegression
```

```
#Dataset
```

```
df=pd.DataFrame({  
    'Area':[1000,1200,1500,1800,2000],  
    'Bedrooms':[2,3,3,4,4],  
    'Age':[10,8,5,4,2],  
    'Price':[300000,350000,450000,500000,600000]  
})
```

```
#features
```

```
X=df[['Area','Bedrooms','Age']] #Using multiple features
```

```
#target
```

```
y=df['Price']
```

```
#model
```

```
model=LinearRegression()
```

```
#Train
```

```
model.fit(X,y) #Learns the relationship btw all features
```

```
#Prediction
```

```
pred=model.predict(X) #predict house price
```

```

#Train
model.fit(X,y)    #learns the relationship btw all features

#Prediction
pred=model.predict(X)    #predict house price

print("Coefficients:",model.coef_)
print("Intercept:",model.intercept_)

```

Coefficients: [-375. 12500. -1250.]

Intercept: 53749.999999999974

What is multicollinearity?

- It occurs when 2 or more independent variables are highly correlated with each other.

Example

Area_sqft	Area_sqm
1000	92.9
1200	111.5

- Both columns contain almost the same information.

Problem ¶

- When features are highly correlated:

1. Model gets confused.

```

print("Coefficients:",model.coef_)
print("Intercept:",model.intercept_)

new=pd.DataFrame({
    'Area':[2500,5448,7000,4999],
    'Bedrooms':[2,1,4,5],
    'Age':[1,45,34,23]})
new_pred=model.predict(new)
print(new_pred)

```

```

Coefficients: [ 250.          -34615.38461538 -13461.53846154]
Intercept: 253846.15384615381
[ 796153.84615385  975461.53846154 1407692.30769231 1020903.84615385]

```

```

[8]: #Multiple regression code
import pandas as pd
from sklearn.linear_model import LinearRegression

#Dataset
df=pd.DataFrame({
    'Area':[100,120,150,180,200],
    'Bedrooms':[3,4,5,6,7],
    'Age':[20,18,12,8,6],
    'Price':[30000,35000,45000,50000,60000]
})

#features
X=df[['Area','Bedrooms','Age']] #Using multiple features

#target
y=df['Price']

#model
model=LinearRegression()

```

1. Model gets confused.
2. Coefficients become unstable
3. Interpretation becomes difficult

How to detect Multicollinearity?

Using

- VIF(Variance Inflation Factor)
- VIF measures how much a feature is correlated with other features.

VIF Interpretation

VIF Value	Meaning
1	No correlation
1 - 5	Moderate correlation
> 5	High correlation
> 10	Severe correlation

#VIF Code

```
import pandas as pd

from statsmodels.stats.outliers_influence import variance_inflation_factor

# Features
X = df[['Area', 'Bedrooms', 'Age']]

# VIF Calculation
vif = pd.DataFrame()

vif['Feature'] = X.columns

vif['VIF'] = [
    variance_inflation_factor(X.values, i)
    for i in range(X.shape[1])]

print(vif)
```

	Feature	VIF
0	Area	165.122797
1	Bedrooms	174.483505
2	Age	2.717105

3. Polynomial Regression

- It is an extension of linear regression that captures non-linear relationships by adding polynomial terms.

Why Polynomial regression?

- Not all relationships are straight lines.

Example:

1. Age vs income
 2. Advertising spend vs sales
 3. Temperature vs electricity consumption
- These relationships often follow curves

Degree selection

Degree determines model complexity

- ----Low degree
 1. Simple model #choose 1
 2. May underfit
- ----High degree #more than 10 or complex
 1. More flexible
 2. May overfit

- ----Optimal degree
1. Best balance
 2. Good generalization on new data

Advantages

- Captures non-linear patterns
- More flexible than linear regression

Disadvantages

- Sensitive to outliers
- Higher risk of overfitting
- Harder to interpret

```
22]: import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

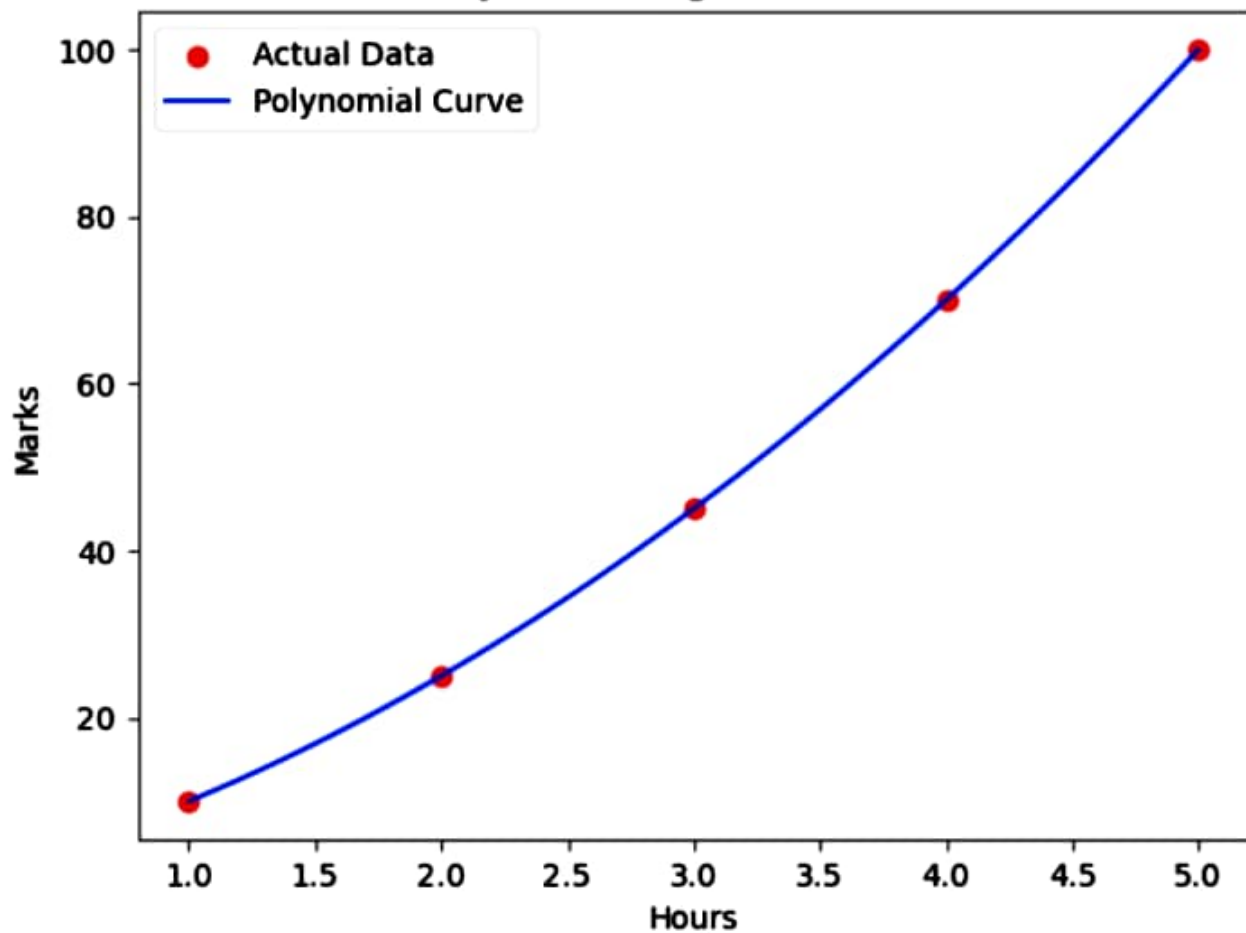
from sklearn.preprocessing import PolynomialFeatures

from sklearn.linear_model import LinearRegression

# Sample dataset
df = pd.DataFrame({

    'Hours': [1, 2, 3, 4, 5],
```


Polynomial Regression Curve



```
'Hours': [1, 2, 3, 4, 5],

'Marks': [10, 25, 45, 70, 100]

})

X = df[['Hours']]
y = df['Marks']

# Polynomial transformation
poly = PolynomialFeatures(degree=4)
X_poly = poly.fit_transform(X)

# Model training
model = LinearRegression()
model.fit(X_poly, y)

# Smooth curve generation (IMPORTANT STEP)
X_range = np.linspace(X.min(), X.max(), 100).reshape(-1, 1)
X_range = pd.DataFrame(X_range, columns=['Hours'])
X_range_poly = poly.transform(X_range)

# Predictions for curve
y_curve = model.predict(X_range_poly)

# Plotting
plt.scatter(X, y, color='red', label='Actual Data')
plt.plot(X_range, y_curve, color='blue', label='Polynomial Curve')

plt.title('Polynomial Regression Curve')
plt.xlabel('Hours')
plt.ylabel('Marks')
plt.legend()
```

Underfitting

- Occurs when the model is too simple.

Characteristics:

- Poor training performance
- Poor testing performance

Trying to fit a straight line to a curved dataset.

Goodfit

- The model captures the underlying pattern without

Overfitting:

- Occurs when the model learns noise instead of actual patterns.

Characteristics

- Excellent training accuracy
- Poor testing accuracy

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression

X = np.array([1,2,3,4,5,6,7,8,9,10]).reshape(-1,1)

y = np.array([2,5,8,15,28,35,50,60,85,95])

# Degree 1
poly1 = PolynomialFeatures(degree=1)
X1 = poly1.fit_transform(X)

# Degree 4
poly4 = PolynomialFeatures(degree=4)
X4 = poly4.fit_transform(X)

# Degree 9
poly9 = PolynomialFeatures(degree=9)
X9 = poly9.fit_transform(X)

model1 = LinearRegression().fit(X1,y)
model2 = LinearRegression().fit(X4,y)
model9 = LinearRegression().fit(X9,y)

plt.scatter(X,y)

plt.plot(X, model1.predict(X1), label="Degree 1")
plt.plot(X, model2.predict(X4), label="Degree 4")
plt.plot(X, model9.predict(X9), label="Degree 9")

plt.legend()
plt.show()
```

- Degree 1 -> Underfitting - straight line misses the pattern
- Degree 4 -> Good fit - Captures the overall curve
- Degree 9 -> Overfitting - Tries to pass through every point - Learns noise instead of pattern

Benefits of regularization

- Reduces overfitting
- Improves model generalization
- handles multicollinearity among features
- Prevents very large coefficient values
- Creates more stable and reliable models

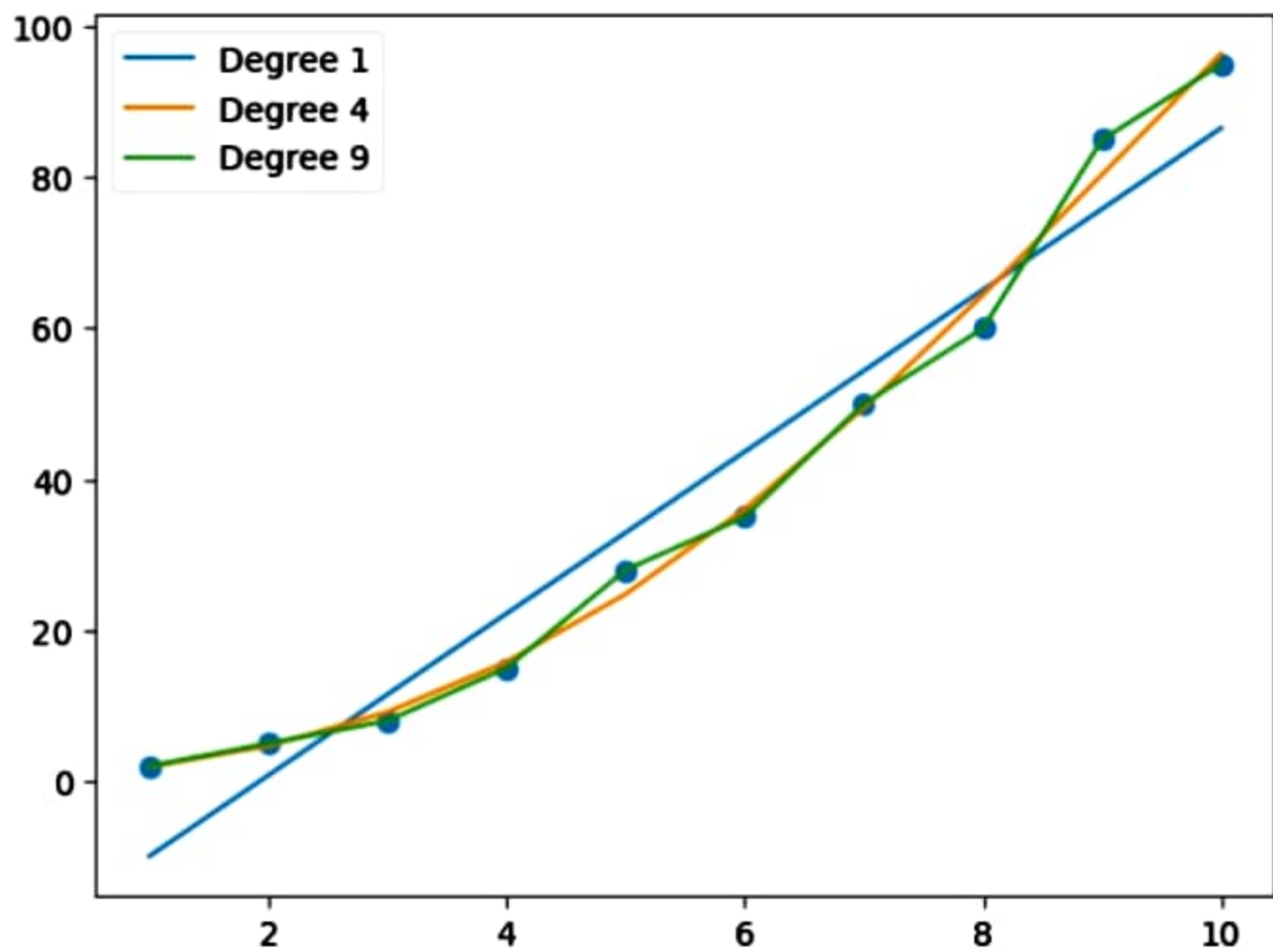
Types

1. Ridge Regression (L2 Regularization)

- Ridge Regression adds the sum of squared coefficients as a penalty term to the cost function.

Characteristics:

- Shrinks coefficients values toward 0.
- Does not make coefficient exactly 0.
- Useful when all features are important.
- Handles multicollinearity effectively.



2. Lasso Regression (L1 Regularization)

- Lasso Regression adds the sum of the absolute coefficient values as a penalty term to the cost function.

Cost function

$$\text{Cost Function} = \text{MSE} + \lambda \sum_{i=1}^n w_i^2$$

Characteristics:

- Shrinks coefficients towards 0.
- Can make some coefficient exactly 0.
- Performs automatic features selection.
- Useful when many irrelevant features exist.

3. ElasticNet Regression

- Elastic combines both Ridge(L2) and Lasso(L1) penalties.

Cost function

$$\text{Cost Function} = \text{MSE} + \lambda_1 \sum |w_j| + \lambda_2 \sum w_j^2$$

Characteristics:

- Combines advantages of Ridge and Lasso.
- Performs Feature selection.
- Handles multicollinearity.
- Suitable for datasets with many correlated features.

##Lasso Code

```
from sklearn.linear_model import Lasso
from sklearn.datasets import make_regression
```

```
X, y = make_regression(
    n_samples=100,
    n_features=10,
    noise=20,
    random_state=42
)
```

```
lasso = Lasso(alpha=2.0)
```

```
lasso.fit(X, y)
```

```
print("Coefficients:")
```

```
print(lasso.coef_)
```

Coefficients:

```
[17.88753444 54.76076704  0.          60.9632585  90.0800089  68.30461063
 81.35787563  4.03519677  2.86610797 67.99906953]
```


When to use each method



Method	When to Use
Ridge (L2)	When all features are important and multicollinearity exists.
Lasso (L1)	When feature selection is required.
Elastic Net	When features are highly correlated and feature selection is also needed.

```
] : ##Ridge Code
from sklearn.linear_model import Ridge
from sklearn.datasets import make_regression

X, y = make_regression(
    n_samples=100,
    n_features=5,      #columns
    noise=20,          #unwanted data
    random_state=42
)

ridge = Ridge(alpha=1.0)      #alpha is for penalty

ridge.fit(X, y)

print("Coefficients:")

print(ridge.coef_)

Coefficients:
[62.4844981  98.07649112  57.01070157  52.31272567  35.35343002]
```

```
##ElasticNet Code
```

```
from sklearn.linear_model import ElasticNet
from sklearn.datasets import make_regression
```

```
X, y = make_regression(
```

```
    n_samples=100,
    n_features=10,
    noise=20,
    random_state=42
```

```
)
```

```
elastic = ElasticNet(
```

```
    alpha=3.0,

    l1_ratio=0.5          #0.0 to 1.0
```

```
)
```

```
elastic.fit(X, y)
```

```
print("Coefficients:")
print(elastic.coef_)
```

Coefficients:

```
[10.65990986 22.10890939  0.49979146 25.9235125  33.08321402 34.26097045
 35.7929364  -7.32756763  4.65692898 25.41452644]
```